

Лабораторная работа №2

Содержание

Организация циклических вычислений в языке C++: оператор цикла <i>while</i>, <i>for</i>	1
Цикл с параметром или цикл типа <i>for</i>	2
Форматирование пересылаемых в поток данных	5
Цикл <i>while</i>	7
Общие требования к выполнению заданий	8
Задание 1.....	8
Метод половинного деления	8
Задание 2.....	8
Варианты для задания 2.....	9
Требования к отчёту.....	12

Организация циклических вычислений в языке C++: оператор цикла *while*, *for*

Алгоритм циклической структуры - это алгоритм, в котором происходит многократное повторение одного и того же участка программы. Такие повторяемые участки вычислительного процесса называются циклами.

Операторы цикла используются для организации многократно повторяющихся вычислений. Любой цикл состоит из тела цикла, то есть тех операторов, которые выполняются несколько раз, начальных установок, модификации параметра цикла и проверки условия продолжения выполнения цикла. Один проход цикла называется итерацией. Проверка условия выполняется на каждой итерации либо до тела цикла (тогда говорят о цикле с предусловием), либо после тела цикла (цикл с постусловием). Разница между ними состоит в том, что тело цикла с постусловием всегда выполняется хотя бы один раз, после чего проверяется, надо ли его выполнять еще раз. Проверка необходимости выполнения цикла с предусловием делается до тела цикла, поэтому возможно, что он не выполнится ни разу.

Переменные, изменяющиеся в теле цикла и используемые при проверке условия продолжения, называются параметрами цикла. Целочисленные параметры цикла, изменяющиеся с постоянным шагом на каждой итерации, называются счетчиками цикла.

Начальные установки могут явно не присутствовать в программе, их смысл состоит в том, чтобы до входа в цикл задать значения переменным, которые в нем используются.

Программа циклической структуры содержит один или несколько циклов. Различают детерминированные циклы с заранее известным числом повторений и итерационные циклы, в которых число повторений заранее неизвестно.

В языке C++ существует 3 вида циклов:

1. цикл с параметром или цикл типа `for`,
2. цикл с предусловием или цикл типа `while`,
3. цикл с постусловием или цикл типа `do ... while`.

Во всех циклах повторяющаяся часть (тело цикла) состоит из одного оператора, если требуется выполнить в цикле несколько операторов, они заключаются в фигурные скобки, образуя составной оператор. Также во всех типах циклов условие продолжения цикла заключается в круглые скобки.

Цикл завершается, если условие его продолжения не выполняется. Возможно принудительное завершение, как текущей итерации, так и цикла в целом. Для этого служат операторы `break`, `continue`, `return` и `goto`.

Цикл с параметром или цикл типа `for`. Для цикла типа `for` заголовок цикла состоит из трех разделов: инициализации (присваивания начальных значений), проверки условия повторения, модификации (изменения параметров). Разделителем между разделами заголовка цикла типа `for` служит точка с запятой.

Основная форма цикла **`for`** имеет следующий вид:

```
for (<список_инициализации>; <условие_продолжения>;  
<переход_к_следующей_итерации>) <тело_цикла>;
```

Все три параметра цикла `for` должны быть разделены точкой с запятой. Перед выполнением цикла выполняются операторы, содержащиеся в первом параметре `<список_инициализации>` оператора `for`, затем осуществляется проверка условия продолжения цикла (обычное условное выражение, определяющее, при каких условиях цикл будет продолжен) в параметре `<условие_продолжения>`, если данное условие принимает *истинное значение*, то выполняются операторы, образующие тело цикла (`<тело_цикла>`). Изменение (приращение) обычно используется для изменения параметра цикла каждый раз при повторении цикла. Если условие продолжения принимает *ложное значение*, то работа цикла завершается, и выполняются операторы, следующие за телом цикла. После выполнения последнего оператора в теле цикла, выполняются `<переход к следующей итерации>` операторы третьего параметра цикла `for`. После чего снова производится проверка условия продолжения и процесс повторяется.

Инициализация используется для объявления и присвоения начальных значений величинам, используемым в цикле. В этой части можно записать несколько операторов, разделенных запятой (операцией «последовательное выполнение»). Областью действия переменных, объявленных в части инициализации цикла, является цикл. Инициализация выполняется один раз в начале исполнения цикла.

Выражение определяет условие выполнения цикла: если его результат, приведенный к типу `bool`, равен `true`, цикл выполняется. Цикл с параметром реализован как цикл с предусловием.

Модификации выполняются после каждой итерации цикла и служат обычно для изменения параметров цикла. В части модификаций можно записать несколько операторов через запятую. Простой или составной оператор представляет собой тело цикла. Любая из частей оператора `for` может быть опущена (но точки с запятой надо оставить на своих местах!).

В качестве параметра цикла необязательно использовать целочисленный счетчик. Далее показан фрагмент программы, выводящей на экран все заглавные буквы английского алфавита:

```
unsigned char ch;
```

```
for (ch = 'A'; ch <= 'Z'; ch++) cout << ch;
```

Следующий фрагмент программы показывает как вывести на печать квадраты вещественных чисел от -2 до 3 с шагом 0.1:

```
double x;
for (x = -2.; x <= 3.; x += 0.1)
    cout << x*x << endl;
```

С помощью цикла типа `for` удобно находить суммы, произведения, искать максимальные и минимальные значения и т.п. При нахождении суммы некоторой переменной, например, `S` присваивается значение 0, затем в цикле к этой переменной прибавляется соответствующий член заданной последовательности. Далее показано, как можно найти сумму натуральных чисел от 1 до 10:

```
int S = 0;
for (int i = 1; i < 11; i++) S += i;
    cout << S << endl;
```

с использованием операции «запятая» и операции постфиксного инкремента данная задача может быть решена следующим образом:

```
for (int S = 0, i = 1; i < 11; S += i++)
    cout << S << endl;
```

Цикл `for` может быть вложенным. В качестве примера рассмотрим программу, печатающую таблицу умножения целых чисел от 0 до 9:

Листинг 1

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
    int i, j;
    for (i = 0; i < 10; i++)
        for (j = 0; j < 10; j++)
            cout << i << " * " << j << " = " << i*j << endl;

    getch();
    return 0;
}
```

Пример 1. Написать программу, позволяющую вычислить сумму квадратов натуральных чисел от 1 до n , где n вводится с клавиатуры.

$$S = 1^2 + 2^2 + \dots + n^2.$$

Листинг 2

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
    int n, i, S = 0;
    system("chcp 1251>null");
    cout << "Введите значение n : ";
    cin >> n;
    for (i = 1; i <= n; i++)
        S += i*i;
    cout << "Сумма чисел S = " << S << endl;
    getch();
    return 0;
}
```

Результат работы программы:

Введите значение n : 5
Сумма чисел S = 55

Пример 2. Написать программу, позволяющую вычислить сумму

$$\sum_{n=1}^N \frac{\log(n * x)}{n^2}$$

где N и x вводятся с клавиатуры.

Листинг 3

```
#include <iostream>
#include <conio.h>
#include <cmath>

int main()
{
    system("chcp 1251>null"); //для visual studio
    double x, S = 0.;
    int k, n;
    std::cout << "Введите значение x : ";
    std::cin >> x;
    std::cout << "Введите значение N : ";
    std::cin >> N;
    for (k = 1; k <= N; k++)
    {
        double a = log(k*x) / (k*k);
        S += a;
    }
    std::cout << "Конечная сумма : " << S << std::endl;
    return 0;
}
```

Очень часто в операторе `for` используются операторы инкремента (`++`) - увеличения на единицу и декремента (`--`) - уменьшения на единицу. Оба оператора используются как в префиксной, так и в постфиксной формах. Префиксная операция инкремента (`++i`) - увеличение на 1 операнда до его использования, соответственно, префиксная операция декремента (`--i`) - уменьшение на 1 операнда до его использования. Постфиксная операция инкремента (`i++`) - увеличение значения операнда на 1 после его использования, соответственно, постфиксная операция декремента (`i--`) - уменьшение значения операнда на 1 после его использования. Операнд этих операций не может быть константой либо другим праводопустимым выражением. Операндом не может быть и произвольное выражение. Операндом унарных операций `++` и `--` должны быть всегда леводопустимые выражения, например, переменные.

Пример 3. Написать программу, позволяющую протабулировать функцию

$$y(x) = \begin{cases} 2x^2 + \cos(x), & \text{если } x \leq 0 \\ x + 1, & \text{если } 0 < x < 5 \\ \sin(2x) - x^2, & \text{если } x \geq 5 \end{cases}$$

в диапазоне от -10 до 10 в равноудаленных точках.

Листинг 4

```

#include <iostream>
#include <conio.h>
#include <cmath>
#include <iomanip>

const double x_min = -10.;
const double x_max = 10.;
const int N = 100;
int main()
{
    system("chcp 1251>null");
    double y, dx = (x_max - x_min) / N;
    for (double x = x_min; x <= x_max; x += dx)
    {
        if (x <= 0.) { y = 2 * x*x + cos(x); }
        else if (x < 5.) { y = x + 1; }
        else { y = sin(2 * x) - x*x; }
        std::cout << "x = " << std::setw(6) << std::setprecision(2) <<
std::fixed << x
        << " \ty = " << std::setw(9) << std::setprecision(5) << y
        << std::endl;
    }
}

```

Любой из трех параметров цикла `for` может быть опущен. Например, найти сумму натуральных чисел от 1 до 10 можно с помощью следующего фрагмента программы:

```

for (int S = 0, i = 1; i < 11;)
{
    S += i;
    i++;
}

```

Если не задано условие завершения в цикле `for`, то он будет выполняться бесконечно. В этом случае для завершения работы цикла необходимо использовать оператор `break`. Для нахождения суммы чисел от 1 до 10 можно использовать также следующий фрагмент кода:

```

for (int S = 0, i = 1; ; i++)
{
    if (i > 10) { break; }
    S += i;
}

```

Также может быть пропущен список инициализации в цикле `for`, в этом случае:

```

int S = 0, i = 1;
for (; i < 11; i++) { S += i; }

```

При выполнении итерационного цикла `for` условие продолжения может изменяться либо при вычислении его значений, либо под действием операторов тела цикла, либо под действием выражений из списка заголовка. Если условие продолжения не изменяется, либо отсутствует, то цикл *бесконечен*. Следующие операторы обеспечивают бесконечное выполнение пустых операторов:

```

for ( ; ; ); // бесконечный цикл
for ( ; 1; ); // бесконечный цикл

```

Форматирование пересылаемых в поток данных. Непосредственное применение операций вывода `<<` (включение в поток) и ввода `>>` (извлечение из потока) к стандартным потокам `cout`, `cin` для данных базовых типов приводит к использованию "умалчиваемых" форматов внешнего представления пересылаемых значений. Например,

при выводе чисел каждое из них занимает ровно столько позиций, сколько необходимо для его представления. Это не всегда удобно и правильно. Например, выполнение операторов

```
int i1 = 1, i2 = 2, i3 = 3, i4 = 4, i5 = 5;
cout << "\n" << i1 << i2 << i3 << i4 << i5;
```

приведет к такому результату:

```
12345
```

Для улучшения читаемости проще всего явно ввести разделительные пробелы. Выполнив оператор

```
cout << '\n' << i1 << ' ' << i2 << ' ' << i3 << ' ' << i4 << ' ' << i5;
```

получим более наглядный результат:

```
1 2 3 4 5
```

Следующий шаг - добавление пояснительного текста и (или) символов табуляции. Ширина (количество позиций) внешнего представления каждого числа выбирается автоматически, исходя из необходимого количества позиций. Единообразие не всегда устраивает пользователя программы. Например, периодическую дробь $1.0 / 3.0$ можно представить весьма различными способами:

```
0.3      0.3333   3.3e-1   0.3333333e0
```

Однако стандартное представление при выводе с помощью оператора

```
cout << "\n1.0 / 3.0 = " << 1.0 / 3.0 << '\n';
```

будет всегда одинаковым (6 знаков после запятой):

```
1.0 / 3.0 = 0.333333
```

Такое поведение выходного потока при использовании операции включения со значением типа `double` предусматривается по умолчанию.

Для управления выводом и вводом предусмотрены специальные функции – манипуляторы. Эти функции - манипуляторы предназначены для использования непосредственно в потоке вывода (между операциями “<<”). Манипуляторы могут иметь параметры и не иметь параметров. Если манипуляторы не имеют параметров, то размещение их в потоке вывода не требует указания круглых скобок (“(” , “)”), используемых обычно для обозначения вызова функции. Этим манипуляторы отличаются от обычных функций, у которых такое недопустимо. Пример манипулятора `endl`, вызывающего перевод строки:

```
std::cout << std::endl; // Эквивалентно выводу символа '\n'
```

Манипуляторы описаны в специальном заголовочном файле `<iomanip>`. Этот файл должен быть включен в программу для использования манипуляторов:

```
#include <iomanip>
```

Основные **стандартные** манипуляторы:

setw – устанавливает ширину поля вывода;

```
cout << setw(30) << sMas << endl; //Строка sMas выводиться в поле шириной 30 позиций
```

setprecision – устанавливает точность вывода;

```
cout << setprecision(3) << f << endl;
```

fixed – для вещественных чисел (типов `float`, `double`) используется представление в формате с фиксированной точкой, причем количество цифр дробной части определяется заданной точностью;

```
cout << fixed << f << endl;
```

left– вывод значения с левым выравниванием (прижать к левому краю поля);

```
cout << left << setw(30) << setfill('!') << sMas << "sMas после left в потоке" << endl;
```

right– вывод значения с правым выравниванием (это значение устанавливается по умолчанию);

```
cout << right << setw(30) << setfill('!') << sMas << "sMas после right в потоке" << endl;
```

endl – переводит строку при выводе (аналогично выводу '\n');

```
cout << endl << endl << endl;
```

С другими стандартными манипуляторами можно познакомиться в литературе.

Цикл **while**.

Основные пункты:

- число повторений инструкций цикла **while** определяется ходом выполнения программы;
- инструкции цикла **while** выполняются до тех пор, пока значение выражения, записанного после слова **while**, не станет равным нулю;
- после слова **while** надо записывать условие выполнения инструкций цикла;
- для завершения цикла **while** в теле цикла обязательно должны быть инструкции, выполнение которых влияет на условие завершения цикла;
- цикл **while** — это цикл с предусловием, т. е. возможна ситуация, при которой инструкции тела цикла ни разу не будут выполнены;
- цикл **while**, как правило, используются для организации приближенных вычислений, в задачах поиска и обработки данных, вводимых с клавиатуры или из файла.

Цикл **while** - это условный цикл, использующий условие на входе (предусловие). Он называется условным, поскольку выполнение его операторной части зависит от истинности условия, представленного условным выражением, таким как, например, (**index<5**). Это выражение является предусловием, поскольку условие должно быть истинным, прежде чем программа начнет выполнять цикл. Цикл с предусловием **while** в общем виде может быть записан:

```
while (условное выражение) { операторы цикла; }
```

Циклы **while**, **for** являются циклами с предусловием. Проверка истинности условия осуществляется перед каждой итерацией цикла, благодаря чему существует вероятность, что операторы, предусматриваемые циклом, никогда не будут выполнены.

Пример: Написать программу, в которой вычисляется сумма последовательности целых чисел до тех пор, пока результат не будет равным 100.

```
#include <iostream>

int main()
{
    system("chcp 1251>nul");
    int i = 1, s = 0;
    while (s <= 100)
    {
        s+=i;
        i++;
    }
    std::cout << "s =" << s << " результат достигнут за " << i << " шагов \n" <<
    std::endl;
    return 0;
}
```

Общие требования к выполнению заданий

Все задания подразумевают работу с вещественными числами. Также каждое отдельное задание должно быть реализовано в отдельной функции. Для вычисления математических функций следует использовать заголовочный файл `cmath`.

При вычислении значения очередного члена ряда используйте значение предыдущего члена, предварительно сохранив его в отдельную переменную (см. [Задание 2](#)).

Обеспечьте вывод данных заданной точности с использованием манипуляторов `std::setw` и `std::setprecision` из заголовочного файла `iomanip`.

Задание 1

Найти корень уравнения: $x - \cos(x) = 0$ методом половинного деления;

Посчитать результат с погрешностью $1e-4$.

Метод половинного деления

Изначально берется отрезок $[x[L], x[R]]$, значений x . Начальные значения $x[L]$ и $x[R]$ вводятся с клавиатуры. Если $f(x[L]) \cdot f(x[R]) < 0$, то для непрерывных функций это будет означать, что искомый x находится на данном отрезке.

Затем находится значение функции на середине этого отрезка $x[M]$. Если значение не является искомым, то отрезок делится на два равных (пополам): $[x[L], x[M]]$ и $[x[M], x[R]]$.

Выбирается тот отрезок, у которого значения функций на концах отрезка имеют противоположные знаки. Для этого отрезка повторяется предыдущее действие.

И так продолжается, пока не будет найден корень уравнения, то есть разница между $x[L]$ и $x[R]$ будет незначительна (меньше погрешности $1e-4$).

Задание 2

Вычислить значение суммы ряда Тейлора для каждого значения x (см. [вариант задания](#)) и сравнить полученные результаты со значением в этом x соответствующей табличной функции. То есть по результатам работы программы на экран должна быть выведена такого плана таблица:

X	Y(x)	S(x)	N
0.000000	0.540302	0.540302	5
0.200000	0.825336	0.825336	4
0.400000	0.980067	0.980067	3
0.600000	0.980067	0.980067	3
0.800000	0.825336	0.825336	4
1.000000	0.540302	0.540302	5

Расчет степенных функций, особенно факториалов, может привести к быстрому росту значений и, вследствие, к потере точности вычислений и даже переполнению. Чтобы этого избежать, следует использовать свойство сходимости ряда Тейлора. Это позволяет вычислять каждый следующий член ряда Тейлора по следующей формуле:

$$A_i = A_{i-1} \cdot k$$

Где A_i - i -ый член ряда Тейлора; k - коэффициент, соотношение вида:

$$k(x, n) = \frac{A_i(x, n)}{A_{i-1}(x, n)}$$

Расчет ряда Тейлора следует заканчивать тогда, когда очередной рассчитываемый член ряда меньше погрешности, заданной в задании (либо разница между двумя соседними членами ряда уже незначительна).

Варианты для задания 2

Вариант задания определяется вариантом делением по модулю варианта студента на 13.

1. Вычислите значения суммы ряда:

$$S(x) = x - \frac{x^3}{3!} + \dots + (-1)^n \frac{x^{2n+1}}{(2n+1)!}$$

и функции $Y(x) = \sin(x)$, где $0 \leq x \leq 1$, с шагом $h = 0.2$. Вычисление суммы ряда Тейлора производите с погрешностью, не превышающей $1e-6$.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

2. Вычислите значения суммы:

$$S(x) = 1 + \frac{x^2}{2!} + \dots + \frac{x^{2n}}{(2n)!}$$

и функции:

$$Y(x) = \frac{e^x + e^{-x}}{2}, \text{ где } 0 \leq x \leq 1, \text{ с шагом } h = 0.2. \text{ Вычисление суммы ряда}$$

Тейлора производите с погрешностью, не превышающей $1e-6$.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

3. Вычислите значения суммы:

$$S(x) = 1 + \frac{\cos \frac{\pi}{4}}{0!} x + \dots + \frac{\cos \frac{\pi}{4}}{n!} x^{n+1}$$

и функции:

$$Y(x) = 1 + x * e^x * \cos\left(\frac{\pi}{4}\right), \text{ где } 0 \leq x \leq 1, \text{ с шагом } h = 0.2. \text{ Вычисление суммы}$$

ряда Тейлора производите с погрешностью, не превышающей **1e-6**.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

4. Вычислите значения суммы:

$$S(x) = 1 - \frac{x^2}{2!} + \dots + (-1)^n \frac{x^{2n}}{(2n)!}$$

и функции:

$$Y(x) = \cos(x), \text{ где } 0 \leq x \leq 1, \text{ с шагом } h = 0.2. \text{ Вычисление суммы ряда}$$

Тейлора производите с погрешностью, не превышающей **1e-6**.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

5. Вычислите значения суммы:

$$S(x) = 1 + 3x^2 + \dots + \frac{2n+1}{n!} x^{2n}$$

и функции:

$$Y(x) = (1 + 2x^2)e^{x^2}, \text{ где } 0 \leq x \leq 1, \text{ с шагом } h = 0.2. \text{ Вычисление суммы ряда}$$

Тейлора производите с погрешностью, не превышающей **1e-6**.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

6. Вычислите значения суммы:

$$S(x) = 1 - x^2 + \frac{x^4}{2!} - \dots + \frac{(-1)^n}{n!} x^{2n}$$

и функции:

$$Y(x) = e^{-x^2}, \text{ где } 0 \leq x \leq 1, \text{ с шагом } h = 0.2. \text{ Вычисление суммы ряда Тейлора}$$

производите с погрешностью, не превышающей **1e-6**.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

7. Вычислите значения суммы:

$$S(x) = x^3 - \frac{x^5}{3} + \dots + (-1)^n \frac{x^{2n+3}}{2n+1}$$

и функции:

$$Y(x) = x^2 \arctg(x), \text{ где } 0 \leq x \leq 1, \text{ с шагом } h = 0.2. \text{ Вычисление суммы ряда}$$

Тейлора производите с погрешностью, не превышающей **1e-6**.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

8. Вычислите значения суммы:

$$S(x) = 1 + \frac{2x}{1!} + \dots + \frac{(2x)^n}{n!}$$

и функции:

$Y(x) = e^{2x}$, где $0 \leq x \leq 1$, с шагом $h = 0.2$. Вычисление суммы ряда Тейлора производите с погрешностью, не превышающей **1e-6**.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

9. Вычислите значения суммы:

$$S(x) = 1 + 2 \frac{x}{2} + \dots + \frac{n^2 + 1}{n!} \left(\frac{x}{2} \right)^n$$

и функции:

$Y(x) = \left(\frac{x^2}{4} + \frac{x}{2} + 1 \right) e^{\frac{x}{2}}$, где $0 \leq x \leq 1$, с шагом $h = 0.2$. Вычисление суммы

ряда Тейлора производите с погрешностью, не превышающей **1e-6**.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

10. Вычислите значения суммы:

$$S(x) = x - \frac{x^3}{3} + \dots + (-1)^n \frac{x^{2n+1}}{2n+1}$$

и функции:

$Y(x) = \arctg(x)$, где $0 \leq x \leq 1$, с шагом $h = 0.2$. Вычисление суммы ряда Тейлора производите с погрешностью, не превышающей **1e-6**.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

11. Вычислите значения суммы:

$$S(x) = 1 - \frac{3}{2}x^2 + \dots + (-1)^n \frac{2n^2+1}{(2n)!} x^{2n}$$

и функции:

$Y(x) = \left(1 - \frac{x^2}{2} \right) \cos(x) - \frac{x}{2} \sin(x)$, где $0 \leq x \leq 1$, с шагом $h = 0.2$. Вычисление

суммы ряда Тейлора производите с погрешностью, не превышающей **1e-6**.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

12. Вычислите значения суммы:

$$S(x) = 1 - \frac{(2x)^2}{2} + \frac{(2x)^4}{24} - \dots + (-1)^n \frac{(2x)^{2n}}{(2n)!}$$

и функции:

$Y(x) = \cos(2x)$, где $0 \leq x \leq 1$, с шагом $h = 0.2$. Вычисление суммы ряда Тейлора производите с погрешностью, не превышающей $1e-6$.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

13. Вычислите значения суммы:

$$S(x) = 2\left(x + \frac{x^3}{3} + \dots + \frac{x^{2n+1}}{2n+1}\right)$$

и функции:

$Y(x) = \ln \frac{1+x}{1-x}$, где $0 \leq x \leq 1$, с шагом $h = 0.2$. Вычисление суммы ряда Тейлора производите с погрешностью, не превышающей $1e-6$.

Результат представить в виде таблицы (без рамок), которая содержит четыре столбца со значениями x , $Y(x)$, $S(x)$ и N , где N - номер последнего слагаемого ряда;

Требования к отчёту

В отчёте должны быть представлены следующие пункты:

1. Постановка задачи;
2. Расчёт коэффициента для второго задания. Можно рассчитать на листе и вставить в отчёт скриншот расчёта;
3. Блок-схема для каждой задачи;
4. Листинг кода;
5. Скриншоты результата работы программы.